Lab 09

Inheritance

Objectives: At the end of this lab session the student should be able to:

- Apply inheritance in Java application development.
- Write Java classes with overridden methods.

Part 1: Problem Solving Exercises (Time: 50 minutes)

Exercise 1

1. Create a new project named Lab09. Create a package named model and a package called test.

2. In the model package create a class called Person that has two data members: id (int), name (String). Add setter/getter methods.

3. Create a new class named Employee that inherits class Person and has one additional data member: noOfChildren.

4. Add setter/getter method.
   According to this the super class is _____ and the subclass is ____.

5. In the *model* package create a new class TestApp with main() method. In the main method, create object of type Employee.

6. Using setter methods, set the object's data as follows:
   id = 123
   name = "Khaled"
   noOfChildren = 2

   Note: that methods setId, setName are not in the *Employee* but inherited from *Person*.

7. Output the name, id and no of children for the above employee. Run your code.

8. In class Employee, add a constructor that receives one parameter (int) to initialize the variable noOfChildren.
   - You will get an error, why?
     To solve this, add a default constructor in the Employee class.

9. Add a constructor to Person that receives two parameters to initialize id and name. Add output statement in this constructor to print "Person constructor is called now".
   - You will get an error. To solve this, add a default constructor in the Person class.

10. Add a constructor in class Employee that receives three parameters: int, String, int to initialize all data members.
    - Can you access the *id*, *name* fields directly in this constructor? Yes [ ] No [ ].

- If no, then how to solve this issue? One solution is to use the setters of these fields. An alternative way is to explicitly call the constructor of the super class, how to do that?
- Use the keyword" *super*" to call the super class's constructor to initialize the data (in a similar fashion to the use of this() to call an overloaded constructor). Also add output statement in this constructor to print "*Employee constructor is called now*".
- Can you make the super class constructor as the last statement? Yes [ ] No [ ].

11. In the main, create a new object as follows

    Employee emp2=new Employee(124, "Ameen"*,1);*

    Run the program, what is the output? And what do you conclude regarding the order of constructors call? _____

12. Change access modifiers of id, name in class Person to **protected** instead of private.
    - Try to access them directly in the Employee class, are they accessible? Yes[ ] No [ ]
    - In the main method, now try to access the id, name of object emp2, are they accessible?

      Yes [ ] No [ ]
    - Notice that AppTest is not a member of the inheritance hierarchy, but it has access to the protected members. Explain. _____

13. Move the AppTest class to the *test* package that you created earlier. Explain why you get errors and how you fix them. _____

14. Create the public method **toString** in class Person that returns person id and name as comma separated values (CSV). Output the details of emp2 using toString().

15. In class Employee, create public method toString to return all Employee's data: id, name, noOfChildren as comma separated values.
    - Having two methods with same name and same signature, one in the super class and one in the sub class is called _____.
    - How to call super class method toString from sub class one? _____

16. Modify access modifier of toString in class Employee to protected instead of public.

    What do you conclude? _____

17. Now restore toString()'s signature to be public. Also make the id and name fields as private and fix all references made to them. Also remove the output statements from the constructors of the Person and Employee classes – no longer needed.

Part 2: Practice Exercises (Time: 70 minutes)

Exercise 2

1- Working on project Lab09, add the three classes below:

```
package model;

public class Vehicle {
        private int wheels;
        private int weight;


        public Vehicle() {
        }

        public Vehicle(int wheels, int weight) {
```

```
		this.wheels = wheels;
		this.weight = weight;
	}

	public int getWheels() {
		return wheels;
	}

	public void setWheels(int wheels) {
		this.wheels = wheels;
	}

	public int getWeight() {
		return weight;
	}

	public void setWeight(int weight) {
		this.weight = weight;
	}
	@Override
	public String toString() {
		String dashes="";
		String className = getClass().getSimpleName();
		for(int i=0; i< className.length();i++)
			dashes+="-";

		return	"\n" + className
				+ "\n" + dashes
				+ "\n    wheels     : "+ wheels
				+ "\n    weight     : "+ weight;
	}
}
```

```
package model;

public class Car extends Vehicle {
	private int passengers;

	public Car() {
	}

	public Car(int wheels, int weight, int passengers) {
		super(wheels, weight);
		this.passengers = passengers;
	}

	public int getPassengers() {
		return passengers;
	}

	public void setPassengers(int passengers) {
		this.passengers = passengers;
	}

	@Override
	public String toString() {
		return super.toString()+"\n    Passengers  : " + passengers;
```

```
        }
}
```

```
package test;
import model.*;
public class VehicleTest {
        public static void main(String a[]) {
                Vehicle bicycle = new Vehicle(2, 20);
                Car sedan = new Car();
                Car toyota = new Car(4, 2000, 5);
                sedan.setWheels(4);
                sedan.setPassengers(4);
                sedan.setWeight(1500);
                System.out.println(bicycle);
                System.out.println(sedan);
                System.out.println(toyota);
        }
}
```

2- After copying and pasting the classes above, run the **VehicleTest** class.

3- Add to the Vehicle class the public method *getAsCSV()* that returns the attributes values as comma separated values. Let the last value in the CSV string be the name of the class preceded with a comma (hint: use Java's getClass().getSimpleName ()). Override this method in the Car class. Display the *bicycle* and *toyota* objects as CSV values.

4- Add a class called **Truck** that inherits from the Vehicle class. Truck objects have the attributes:

- number of wheels (wheels)
- truck weight (weight)
- load capacity (load)

and the methods:

- no-argument constructor
- parameterized constructor
- getWheelLoad(): to return wheel load which is equal to the *double* quantity (*don't forget casting before division)*:

```
(load + weight )/ wheels
```

- toString() which overrides the superclass method by adding the Truck details and the wheel load to the string generated by the superclass. Let the returned string be properly formatted.
- Override getAsCSV() to return all the values of a truck object as CSV (do not include the wheel load).

5- Add to the main method in the VehicleTest class the code that performs the following:

- Create a Truck object called *volvo* with the data:
  - Number of wheels = 12
  - Truck weight = 4000 kg
  - Load capacity = 12000 kg
- Display the *volvo* data using both toString() and getAsCSV().

Exercise 3

a) In the test package create a new class called **PersonTest** which contains a main() method that tests the `Person` class by doing the following:

- Create a List of `Person` instances, each element initialized with the following data:

Person id `(id)`	Person name `(name)`

2571	Ghanim Salih
1318	Saad Mohamed
5376	Adil Salman

Write statements to output the contents of the 3 instances, and then save your file and run it.

Additional Practice Exercises

Exercise 4

1. In the project named **Lab09**, and in the model package create a class called Student that inherits from the Person class. A student has three data members: id (int), name (String) and major (String).

2. Add setter/getter method for the additional attribute. Also add toString() method that returns all the student details as a String.

3. In the AppTest class add two Student instances.

4. Display the details of the two students you just created.